CMPT 470 Milestone 2 Report

Future Business Majors

Assessing Software Quality Using Static Analysis

Kylen Morris (gth842)

Darian Rowat (ddr271)

Rylan Curtis (ody435)

Vlad Akulov (onc736)

Kyran McKnight-Woods (zoe821)

January 30, 2026

1. Introduction

This paper discusses the setup, execution and analysis of several given static analysis tools on a given repository. Static analysis tools are used to inspect and quantify code quality without any execution of code. Different analysis tools provide different insights, from coding standards and maintainability, semantic patterns used to identify bugs, potential security vulnerabilities, to structural complexity highlighting potential decision hotspots in code. These tools are useful for software engineers to ensure potential errors, vulnerabilities, and inconsistencies are flagged, resulting in more readable, maintainable, and reliable code. The goal of this lab and milestone will be to learn how these tools work, how to interpret their results, and what their strengths are, rather than giving an in-depth analysis of the given repository.

Overview of GitHub repository

The given repository we analysed here consists of a fully featured http client library for python3. The library supports http1.1 and http2 and provides both blocking and nonblocking APIs. The repo itself is a library so it is framework agnostic by design. The main tool of the library is the request API that the tool relies on, and the underlying networking layout is copied from urllib3. The repository seems robust and mature with lots of proper documentation, pull requests, users, contributors and forks. Given this information we expected to find few structural issues with the possibility of stylistic/personal decisions from the creators and maintainers.

Overview of the tools used

The static analysis tools we used provide reports containing information about potential bugs, security vulnerabilities, and style violations or complexities. We used three different analysis tools. The first tool we used was Semgrep, which is a lightweight, rule-based static analysis tool that can detect bug patterns, insecure practices, and maintainability issues using syntactic and semantic rules. The second tool we used is Pylint, which analyzes Python source code to detect potential errors and maintainability concerns which are then sorted by severity and type. Finally, we used Radon which identifies structural complexity hotspots at the function and module level.

2. Experimental Setup

We had a wide mix of hardware amongst our group members, so we spent some time standardising our tools to run on our systems consistently. We solved this problem in a few ways such as using python for a scripting language and using a virtual environment to make sure package versions and python versions were standardised across the board. We also utilised bash scripts to make the process easier to get started with.

The repository we had to analyse was https://github.com/encode/httpx. This repository was quite organized and had all of the python source code in a single folder (/httpx/httpx), resulting in a straightforward setup for running the semantic tools, with only a single subdirectory in this folder.

Our first analysis was conducted using Semgrep which is made with python and thus required a virtual environment to be consistent across machines. Once we were finished, we created a `requirements.txt` by pip freezing the required packaged inside. We used Python version 3.8 for all tools. To time how long Semgrep took, we encapsulated the command using the built in Python time utility so that we could know exactly how much processor time each static analysis tool took without interference from the OS.

For our second analysis, we used Pylint which had had nearly identical methodology with Pylint version 1.5.4. We once again froze our requirements into a requirements.txt file and left it in the repo. To time this we once again used time in the shell so that the runtime was fair for all tools. Pylint's output is quite verbose and is difficult to parse by hand, at least on a codebase of this size. To help us analyze it, we created an `analyze.py` script. This script takes the raw Pylint JSON and runs some basic analysis like totaling each error type and finding the most problematic files.

Lastly, the Radon analysis was almost a direct copy of our Pylint analysis. We once again had the same environment set up. We used version 6.0.1 to conduct our analysis but this time the output was in txt rather than JSON so the Python script to aggregate the results parsed a text file rather than a JSON file.

3. Results

Table 1: Timing Summary

Tool	Time to Analyze
Semgrep	8.039s
Pylint	1.811s
Radon	1.258s

After running our initial analysis and timing all the tools, we can see that Semgrep was by far the longest running. This is likely due to the nature of it, being more than a linter which checks for certain rules and errors. Pylint took almost two seconds, and Radon was the quickest to execute at one and a quarter seconds.

Table 2: Pylint Analysis Summary

Error Type	Count	Total % of Output
Convention	183	47.2%
Refactor	126	32.5%
Warning	58	15.0%
Error	21	5.4%

Here we can see a breakdown of errors provided to us by Pylint, it is worth noting that a considerable number of these errors are convention/refactor hints, with only 20% of the errors being possible sources of serious concern.

Table 3: Semgrep Analysis Summary

Error Type	Count
dangerous-globals-use	1
insecure-hash-algorithms	1

Following up with the Semgrep results we notably only got two warnings despite trying to find more. The repository robustness/maturity is certainly showing here, as well as Semgrep's focus on critical vulnerabilities and rule matching.

Table 4: Radon Analysis Summary

Grade	Count	Total %
A	462	88.5%
B	52	10.0%
C	7	1.3%
D	0	0.0%
E	0	0.0%
F	1	0.2%

Finally, running Radon for complexity, we see that most of the repository is in great shape with one notable exception we will get into later.

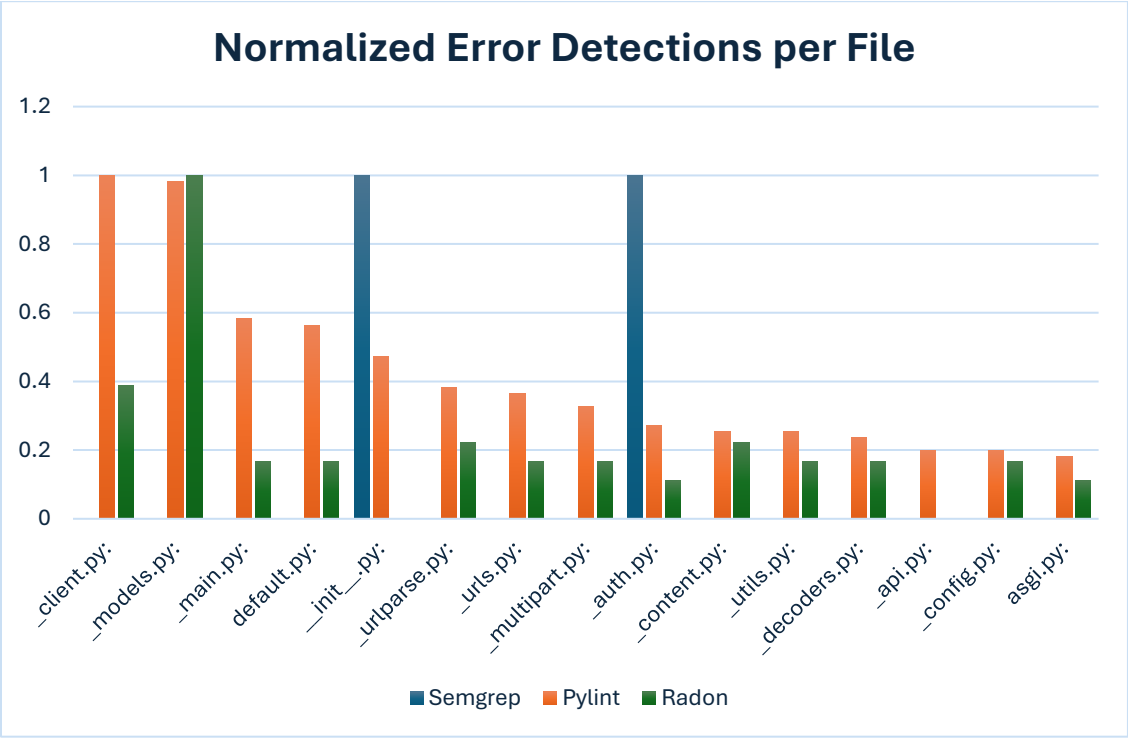

Figure 1: Normalized Tool Detections per File

We took all our error/warning counts per file and normalized them to plot them together. We can see here there is a slight correlation between Radon/Pylint flags despite looking for different things. The base client had the highest number of flags in Pylint, but Pylint and Radon both flagged models as a complex/unconventional file.

4. Analysis and Discussion

Pylint produces stylistic warnings which generally encompass errors based on convention. While these might affect readability, they aren't critical issues with the code. Some warnings under refactor, such as duplicate-code or too-many-positional-arguments, may be indicative of hard to maintain code. Complex hard-to-read code also makes code much less maintainable by making it harder to debug and could result in needing to change the same things in multiple places which is time consuming and error prone.

Some 'warnings' are in a similar boat. Unused parameters, unused imports, and other similar warnings are not essential to maintainable code but are worth looking at. Others, such as arguments-differ and unreachable, are more likely to be a problem. If code is unreachable, it could indicate a missing function call or improper conditional statement. Differing arguments can cause confusion in similar functions and should be standardized to ensure maintainable code.

Radon produced warnings such as unsubscriptable-object which indicates square brackets used to index a non-indexable type or not-an-iterable which indicates trying to iterate over a single item, almost always point to critical errors that are very likely to cause the compiler to fail. Many of the errors we found were import-errors, which may be easier to catch and may not be as much of an issue.

Semgrep found two issues the first being dangerous-globals-use and the second being insecure-hash-algorithms. Both are in theory critical issues which should be looked over and rectified as soon as possible. Taking an actual look at the code, we can see the issues flagged are benign probably because the maintainers have run Semgrep before and fixed the real issues flagged by Semgrep.

Out of the tools used Semgrep produces the most pressing critical and security issues while Pylint and radon are more focused on the “maintainability” and “readability” of the code. They generate errors that are more likely to cause errors down the line while Semgrep produces issues that could be an issue at this very second.

When used in conjunction with one another these tools produce great results for diagnosing and predicting code flaws in advance. All three tools struggle with architectural awareness they cannot assess higher-level system design such as authentication models and trust boundaries. They also struggle due to lack of runtime context because they are all static, they often miss issues the only appear during runtime such as race conditions and resource exhaustion.

5. Lessons Leaned

Static analysis tools are great for early development analysis to help work out smaller bugs and catch design mistakes early on. They are relatively efficient with certain tools being faster than others as well as offering many insights into optimizing readability and correctness.

The biggest downside to static analysis tools is their inability to detect run time conditions and the high false positive rate. Since the severity of some warnings are often subjective, reporting everything can lead to a messy and confusing report causing wasted time checking all outputs. The inability to detect run time conditions also means that static analysis tools are unable to optimize the speed or resource utilization of a program.

Another drawback to static analysis tools is their lack of understanding specific environment constraints. These tools cannot detect the versions of each library and their compatibility with hardware, firmware, or software on any given machine.

Generally, using static analysis tools seems most appropriate for early touch ups, and to ensure correctness as the codebase grows. Once correctness is verified, any further optimizations regarding other performance metrics will likely not benefit much from using these tools.

6. Conclusion

In conclusion, we analyzed a given Python repository (httpx) with several semantic analysis tools. We found that each of these tools has their strengths and weaknesses, and all are useful in some way when developing. These tools (Semgrep, Pylint, and Radon) are extremely fast and effective tools for flagging potential errors, copied code, vulnerabilities, and inconsistencies. Implementing these tools into a workflow is a great way to sanity check your codebase, though they are not perfect.

Pylint has extremely thorough checks for documentation, copied code, and function calling errors, and much more. Radon is a strong tool for flagging potential complexity hotspots in your code which you may want to split up or clearly document. Finally, Semgrep is a slower, but great tool to check for known vulnerabilities and poor security practices. Its rule matching checks for code that other semantic analysis tools may not flag, as it is technically correct code.

Semantic analysis is very fast as you are not running any code, but this leaves some major areas not covered. Code flow, algorithm efficiency, data structure consistency, organization, and many other components of a codebase are not covered. While these tools are great to keep in your toolkit, they do not replace proper unit testing, code tracing, and detailed documentation. Maintaining a strong codebase requires work, and these tools are useful, but not enough on their own.

7. References

[1] encode, "GitHub - encode/httpx: A next generation HTTP client for Python.," *GitHub*, Dec. 06, 2024. https://github.com/encode/httpx (accessed Jan. 30, 2026).

[2] Lacchia, M., "Welcome to Radon's documentation! — Radon 4.1.0 documentation," *Radon*. https://Radon.readthedocs.io/en/latest/ (accessed Jan. 30, 2026).

[3] Logilab and MohriCorporation, "Pylint User Manual — Pylint 1.5.4 documentation," *Pylint*, 2025. https://docs.Pylint.org/ (accessed Jan. 30, 2026).

[4] Semgrep, Inc., "Docs home | Semgrep," *Semgrep, Inc*. https://Semgrep.dev/docs/ (accessed Jan. 30, 2026).